

Министерство образования и науки РФ  
Санкт-Петербургский политехнический университет Петра Великого  
Институт компьютерных наук и кибербезопасности  
Высшая школа программной инженерии

**ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ**  
по дисциплине «Языки моделирования и описания цифровой аппаратуры»

**Программная на SystemC и аппаратная на Verilog HDL  
реализация алгоритма  
«Счётчик последовательности из нулей»**

Выполнил:  
Студент гр. 5130904/30002

Кладковой М.Д.

Проверил:

Амосов В.В.

Санкт-Петербург  
2025 г.

## Оглавление

|                                                               |    |
|---------------------------------------------------------------|----|
| 1. Алгоритм работы счётчика последовательности из нулей.....  | 3  |
| 2. Программная реализация на SystemC.....                     | 5  |
| 2.1. Блок-схема.....                                          | 5  |
| 2.2. Код программы на языке SystemC.....                      | 6  |
| ZeroCounter.h.....                                            | 6  |
| ZeroCounter.cpp.....                                          | 7  |
| ZeroCounterTest.h.....                                        | 7  |
| ZeroCounterTest.cpp.....                                      | 8  |
| Main.cpp.....                                                 | 8  |
| 2.3. Тестирование симуляцией.....                             | 9  |
| 3. Аппаратная реализация на Verilog HDL.....                  | 11 |
| 3.1. Блок-схема.....                                          | 11 |
| 3.2. Код на языке Verilog транслированный с помощью sc2V..... | 12 |
| 3.3. Тестирование симуляцией.....                             | 15 |
| 3.4. Проверка на синтезируемость.....                         | 15 |

# 1. Алгоритм работы счётчика последовательности из нулей

## 20. Счётчик последовательности из нулей

На вход устройства подаётся последовательность бит. Устройство должно подсчитать, сколько нулей было подано. Подача единицы сбрасывает счётчик.

### Входные сигналы:

```
bool in;
```

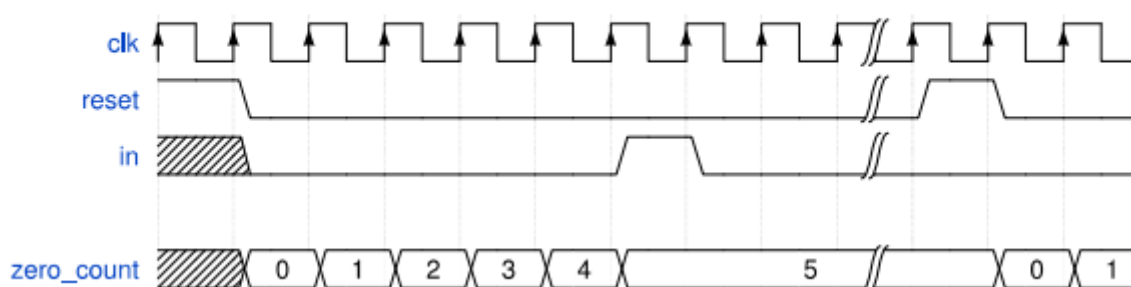
### Выходные сигналы:

```
sc_uint<32> zero_count;
```

### Логика:

последовательная и комбинационная

### Пример временной диаграммы:



### Сигналы:

*clk* — тактовый сигнал

*in* — входной бит:

0 → продолжаем считать,

1 → останавливаем счёт (*freeze*).

*reset* — активный высокий сигнал сброса

*zero\_count* — количество считанных нулей подряд (до первой единицы).

### Алгоритм работы:

#### 1. После запуска/сброса:

- Счётчик обнуляется.
- Разрешается счёт (*running* = true).

#### 2. На каждом такте:

- Если *reset* == 1 → обнуление и разрешение счёта.
- Если *running* == true:
  - Если *in* == 0 → увеличить *count\_reg* на 1.
  - Если *in* == 1 → остановить счёт (*running* = false).

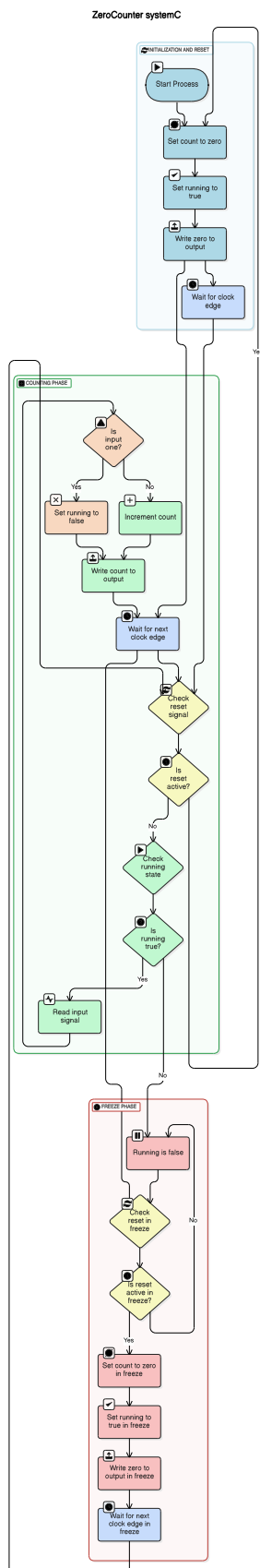
- Выводится текущее значение счётчика на `zero_count`.

### **3. После остановки (`running == false`):**

- Значение счётчика больше не меняется до следующего сброса.

## 2. Программная реализация на SystemC

### 2.1. Блок-схема



## 2.2. Код программы на языке SystemC

### ZeroCounter.h

```
// ZeroCounter.h
#ifndef ZEROCOUNTER_H
#define ZEROCOUNTER_H

#include <systemc.h>

SC_MODULE(ZeroCounter) {
    // INPUTS
    sc_in_clk clk;
    sc_in<bool> reset;
    sc_in<bool> in;           // data bit: 0→count, 1→freeze

    // OUTPUT
    sc_out<sc_uint<32>> zero_count;

private:
    sc_uint<32> count_reg;
    bool running;           // true while counting, false after
    freeze

    void process_count();

public:
    SC_CTOR(ZeroCounter) : count_reg(0), running(true) {
        SC_CTHREAD(process_count, clk.pos());
        reset_signal_is(reset, true);
    }
};

#endif
```

## ZeroCounter.cpp

```
#include "ZeroCounter.h"

void ZeroCounter::process_count() {
    // reset behavior
    count_reg = 0;
    running = true;
    zero_count.write(count_reg);
    wait();
    while (true) {
        if (reset.read()) {
            count_reg = 0;
            running = true;
        } else if (running) {
            if (in.read() == 1) {
                running = false;
            } else {
                count_reg = count_reg + 1;
            }
        }
        zero_count.write(count_reg);
        wait();
    }
}
```

## ZeroCounterTest.h

```
#ifndef ZEROCOUNTERTEST_H
#define ZEROCOUNTERTEST_H

#include <systemc.h>

SC_MODULE(ZeroCounterTest) {
    sc_in_clk clk;
    sc_out<bool> reset;
    sc_out<bool> in;
    sc_in<sc_uint<32>> zero_count;

    void generate_stimuli();

    SC_CTOR(ZeroCounterTest) {
        SC_THREAD(generate_stimuli);
        sensitive << clk.pos();
    }
};
```

```
#endif // ZEROCOUNTERTEST_H
```

## ZeroCounterTest.cpp

```
#include "ZeroCounterTest.h"
#include <iostream>

void ZeroCounterTest::generate_stimuli() {
    // Single-cycle reset at start
    wait();
    reset.write(1);
    wait();
    reset.write(0);
    wait();

    int tests[] = {10, 5, 100, 37};
    for (int idx = 0; idx < 4; ++idx) {
        int n = tests[idx];
        // Drive n zeros
        for (int i = 1; i < n; ++i) {
            in.write(0);
            wait();
        }
        // Single-cycle high pulse on in to freeze
        in.write(1);
        wait();
        // Return in low for hold (doesn't affect running=false)
        in.write(0);
        // Wait 5 cycles before reset
        for (int i = 0; i < 5; ++i) wait();
        // Report
        std::cout << "@" << sc_time_stamp()
                  << ": Expected zero_count=" << n
                  << ", Observed=" << zero_count.read() << std::endl;

        // Single-cycle reset
        reset.write(1);
        wait();
        reset.write(0);
        wait();
    }
    sc_stop();
}
```

## Main.cpp

```
#include <systemc.h>
#include "ZeroCounter.h"
#include "ZeroCounterTest.h"

int sc_main(int argc, char* argv[]) {
    sc_clock clk("clk", 10, SC_NS);

    sc_signal<bool> reset_sig;
    sc_signal<bool> in_sig;
    sc_signal<sc_uint<32>> count_sig;

    ZeroCounter zc("ZeroCounter");
    ZeroCounterTest tb("ZeroCounterTest");

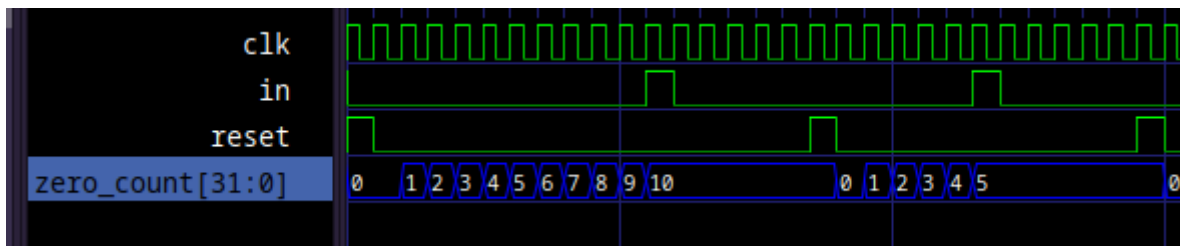
    // Bind ports
    zc.clk(clk);
    zc.reset(reset_sig);
    zc.in(in_sig);
    zc.zero_count(count_sig);

    tb.clk(clk);
    tb.reset(reset_sig);
    tb.in(in_sig);
    tb.zero_count(count_sig);

    // Trace
    sc_trace_file* tf =
sc_create_vcd_trace_file("zero_counter_tb");
    tf->set_time_unit(1, SC_NS);
    sc_trace(tf, clk, "clk");
    sc_trace(tf, reset_sig, "reset");
    sc_trace(tf, in_sig, "in");
    sc_trace(tf, count_sig, "zero_count");

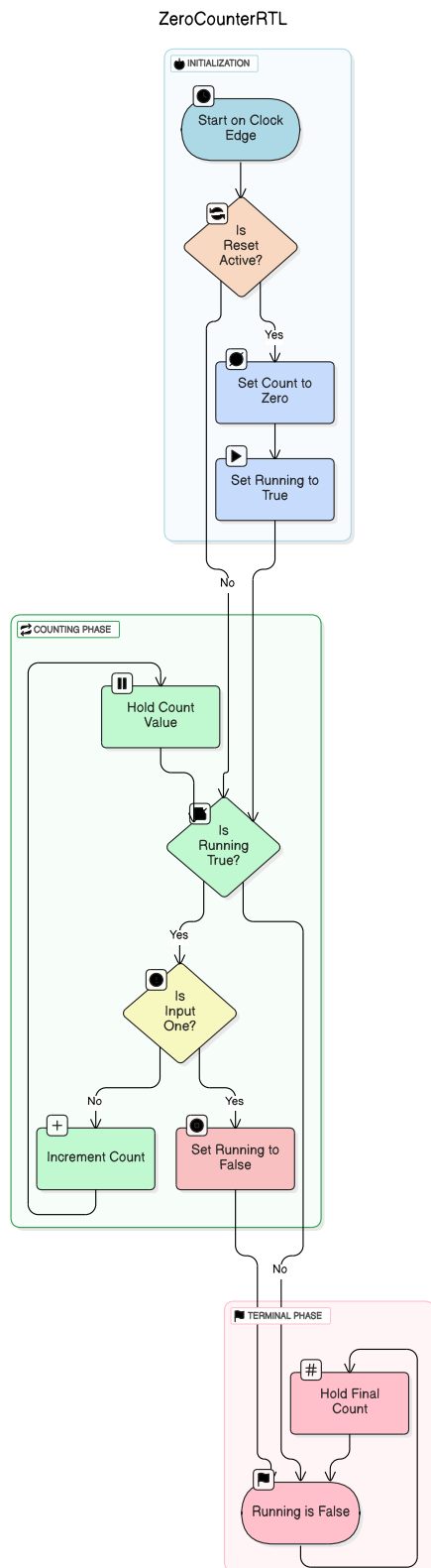
    sc_start(20000, SC_NS);
    sc_close_vcd_trace_file(tf);
    return 0;
}
```

## 2.3. Тестирование симуляцией



### 3. Аппаратная реализация на Verilog HDL

#### 3.1. Блок-схема



## 3.2. Код на языке Verilog транслированный с помощью sc2V

### ZeroCounterRTL.h

```
#ifndef ZEROCOUNTERRTL_H
#define ZEROCOUNTERRTL_H

#include <systemc.h>

/**
 * ZeroCounterRTL
 *
 * * Счетчик количества нулевых тактов до первого единичного
 * входа.
 * * Синтезируемая RTL-реализация на SystemC.
 */
SC_MODULE(ZeroCounterRTL) {
    // Порты
    sc_in_clk      clk;           // тактовый сигнал
    sc_in<bool>     reset;         // асинхронный сигнал сброса
    (активен high)
    sc_in<bool>     in;           // входной бит: 0 → считаем,
    1 → останавливаем счет
    sc_out<sc_uint<32>> zero_count; // выход: текущее значение
    счетчика

    // Внутренние сигналы для хранения состояния
    sc_signal<sc_uint<32>> count_reg, next_count_reg;
    sc_signal<bool>      running,      next_running;

    // Комбинаторная логика: вычисление следующего состояния
    void comb_logic();

    // Последовательная логика: обновление регистров и выходов
    void seq_logic();

    // Конструктор
    SC_CTOR(ZeroCounterRTL) {
        SC_METHOD(comb_logic);
        sensitive << count_reg << running << in << reset;

        SC_METHOD(seq_logic);
        sensitive << clk.pos();
    }
};
```

```
#endif
```

## ZeroCounterRTL.cpp

```
#include "ZeroCounterRTL.h"

void ZeroCounterRTL::comb_logic() {
    sc_uint<32> cnt      = count_reg.read();
    bool        run      = running.read();
    sc_uint<32> cnt_next;
    bool        run_next;

    if (reset.read()) {
        cnt_next = 0;
        run_next = true;
    } else if (run) {
        if (in.read()) {
            cnt_next = cnt;
            run_next = false;
        } else {
            cnt_next = cnt + 1;
            run_next = true;
        }
    } else {
        cnt_next = cnt;
        run_next = run;
    }

    next_count_reg.write(cnt_next);
    next_running.write(run_next);
}

void ZeroCounterRTL::seq_logic() {
    if (reset.read()) {
        count_reg.write(0);
        running.write(true);
    } else {
        count_reg.write(next_count_reg.read());
        running.write(next_running.read());
    }
    zero_count.write(count_reg.read());
}
```

## Verilog

```

module ZeroCounterRTL (
    input        clk,
    input        reset,
    input        in,
    output [31:0] zero_count
);

    reg [31:0] count_reg;
    reg        running;

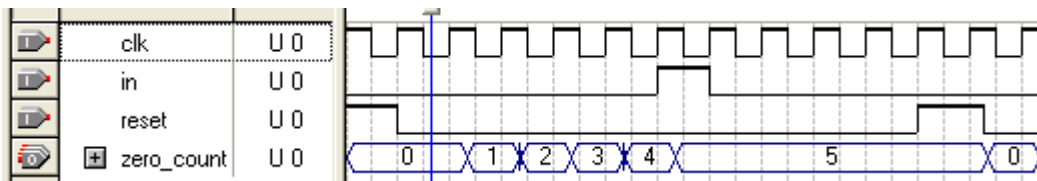
    wire [31:0] next_count;
    wire        next_running;

    assign next_count = reset
        ? 32'd0
        : (running
            ? (in ? count_reg
                  : count_reg + 1)
            : count_reg);
    assign next_running = reset
        ? 1'b1
        : (running
            ? (in ? 1'b0
                  : 1'b1)
            : running);

    always @(posedge clk) begin
        if (reset) begin
            count_reg ≤ 32'd0;
            running   ≤ 1'b1;
        end else begin
            count_reg ≤ next_count;
            running   ≤ next_running;
        end
    end
    assign zero_count = count_reg;
endmodule

```

### 3.3. Тестирование симуляцией

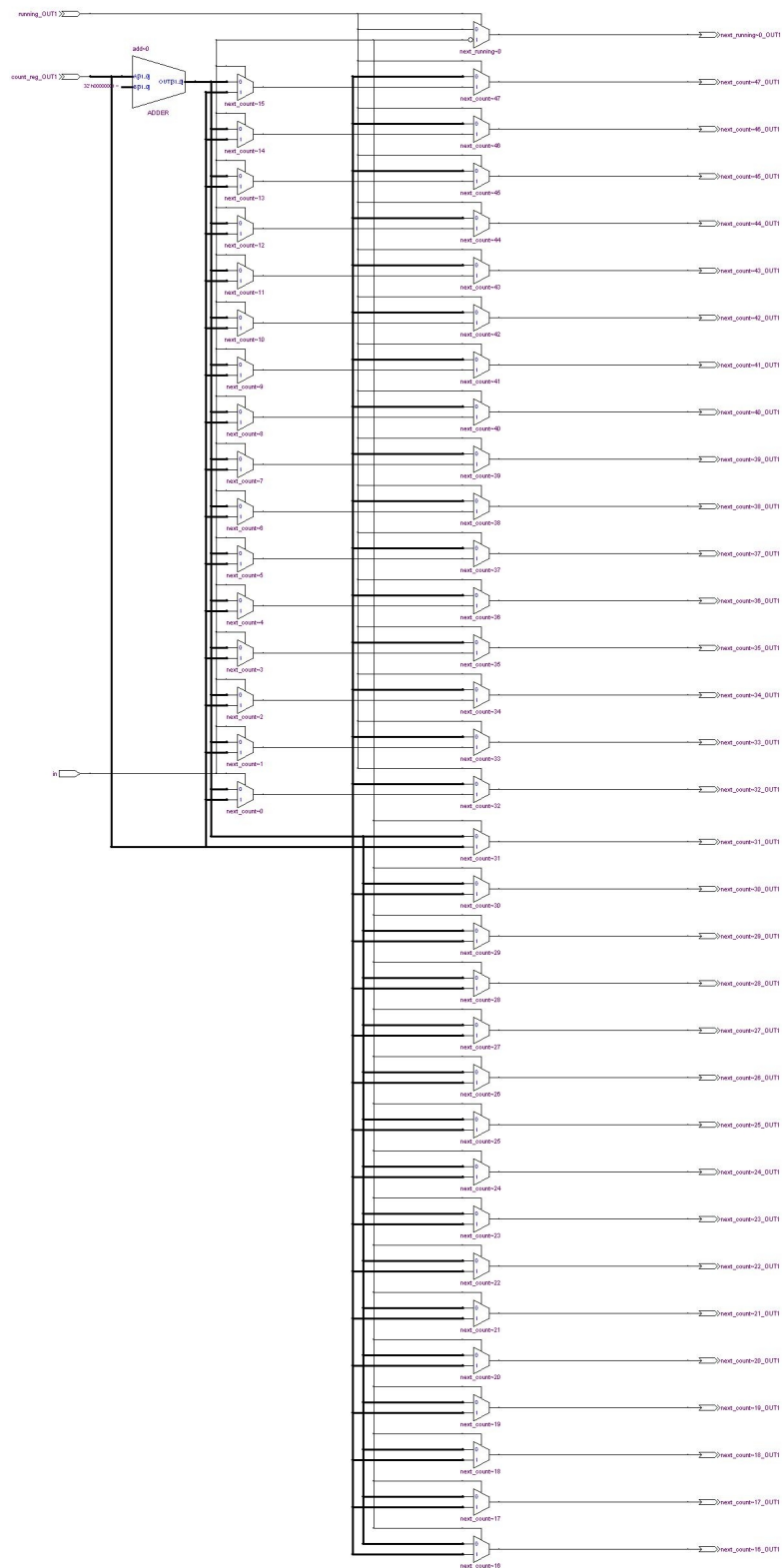


### 3.4. Проверка на синтезируемость

#### Отчет в среде Quartus II

|                          |                                          |
|--------------------------|------------------------------------------|
| Flow Status              | Successful - Fri May 28 12:53:33 2021    |
| Quartus II Version       | 5.0 Build 148 04/26/2005 SJ Full Version |
| Revision Name            | ZeroCounter                              |
| Top-level Entity Name    | ZeroCounterRTL                           |
| Family                   | Stratix                                  |
| Met timing requirements  | Yes                                      |
| Total logic elements     | 34 / 10,570 ( < 1 % )                    |
| Total pins               | 35 / 336 ( 10 % )                        |
| Total virtual pins       | 0                                        |
| Total memory bits        | 0 / 920,448 ( 0 % )                      |
| DSP block 9-bit elements | 0 / 48 ( 0 % )                           |
| Total PLLs               | 0 / 6 ( 0 % )                            |
| Total DLLs               | 0 / 2 ( 0 % )                            |
| Device                   | EP1S10F484C5                             |
| Timing Models            | Final                                    |

**RTL схемы**







## Технологическая схема

